DECEMBER 15, 2025

GROUP MEMBER NAME	GROUP MEMBER REG #
Muhammad Huzaifa	FA23-BCS-113
Awais Ali Asad	FA23-BCS-022
Areeb Ahmed	FA23-BCS-017
Abdullah Nasir	FA23-BCS-007

PROJECT REPORT

AI AUTOMATED HOME SERVICE ROBOT

ARTIFICIAL INTELLIGENCE

MAM FARHA SAEED

Table of Contents

1. Introduction 2
2. Project Objectives 2
3. System Overview & Operational Flow (A–Z) 2
3.1 Environment Initialization 2
3.2 User Interaction Modes 3
3.3 Backend Processing 3
3.4 Pathfinding and Execution 3
4. Detailed Function Call Flow (Manual Task Execution) 4
5. Pathfinding Algorithms (AI Search Module) 5
5.1 Breadth-First Search (BFS) 5
5.2 Depth-First Search (DFS) 5
5.3 Uniform Cost Search (UCS) 5
5.4 Greedy Best-First Search 5
5.5 A* Search (Recommended) 5
Algorithm Comparison 6
6. Intelligent Reasoning & Scheduling Modules 6
6.1 Knowledge Base (Propositional Logic) 6
6.2 Constraint Satisfaction Problem (CSP) 6
6.3 Automated Execution 7
7. Battery Management System 7
8. Graphical User Interface (GUI) 7
9. Output Screenshots 8
10. File Structure Summary 10
11. Conclusion 10

AI Automated Home Service Robot

1. Introduction

The **Home Service Robot** is a simulated intelligent robotic system designed to demonstrate core concepts of **Artificial Intelligence** within a practical and visually rich environment. The project integrates **Search Algorithms**, **Automated Planning**, and **Constraint Satisfaction Problems (CSP)** into a unified architecture, supported by a modern **Dark Theme Graphical User Interface (GUI)**.

This simulation models how an autonomous home robot can perceive its environment, reason about tasks, plan optimal actions, and execute them while respecting constraints such as obstacles, task priorities, and battery limitations.

2. Project Objectives

The primary objectives of this project are:

- To simulate intelligent navigation using classical AI search algorithms.
- To demonstrate task decomposition and automated planning.
- To apply Constraint Satisfaction Problems (CSP) for intelligent task scheduling.
- To visualize robot behavior in real time through a responsive GUI.
- To model real-world constraints such as limited battery resources.

3. System Overview & Operational Flow (A–Z)

The project follows a structured end-to-end execution flow, from initialization to task completion.

3.1 Environment Initialization

- The GUI initializes a **10×10 grid-based environment**.
- Predefined **locations** (Kitchen, Bedroom, Charging Station, etc.) are placed on the grid.
- **Obstacles** (walls) are added to restrict movement.
- The robot is initialized at a fixed starting position (e.g., (0, 9)).

3.2 User Interaction Modes

The system supports two distinct modes of operation:

Mode A: Manual Task Execution

- The user selects a **Pickup Location** and a **Drop-off Location**.
- A pathfinding **algorithm** (BFS, DFS, UCS, Greedy, or A*) is selected.
- The user initiates execution using the **"EXECUTE SINGLE TASK"** button.

Mode B: Intelligent Task Scheduling (CSP-Based)

- The user selects multiple high-level tasks (e.g., Clean House, Serve Tea, Recharge Battery).
- The **"PRIORITIZE & EXECUTE"** button triggers intelligent scheduling.
- The system automatically determines the optimal task order using CSP.

3.3 Backend Processing

- In **Manual Mode**, the planner executes a fixed task sequence:
    - Navigate → Pick → Navigate → Drop.
- In **CSP Mode**, the system:
    - Solves the scheduling problem.
    - Generates an ordered plan.
    - Executes tasks sequentially without further user input.

3.4 Pathfinding and Execution

- For every navigation task, the planner invokes the selected **search algorithm**.
- The algorithm computes an optimal or feasible path while avoiding obstacles.
- The robot follows the computed path step-by-step.
- The GUI updates in real time with animations and path visualization.

4. Detailed Function Call Flow (Manual Task Execution)

The following sequence describes what happens internally when the user clicks **"EXECUTE SINGLE TASK"**.

Step	Function	File	Purpose
1	execute_task_thread()	gui.py	Handles button click in a separate thread for responsiveness
2	execute_task()	gui.py	Collects user input and initializes planner
3	execute_subtask()	planner.py	Starts first subtask (navigate to pickup location)
4	locations.get()	environment.py	Converts location name to grid coordinates
5	a_star_search()	search.py	Computes optimal path to target
6	heuristic()	search.py	Estimates Manhattan distance to goal
7	get_neighbors()	search.py	Expands valid neighboring cells
8	is_free()	environment.py	Validates movement (no wall, within grid)
9	reconstruct_path()	search.py	Builds final coordinate path
10	move()	robot.py	Updates robot position
11	update_grid()	gui.py	Redraws GUI with updated robot position
12	pick_item()	robot.py	Simulates item pickup
13	execute_subtask()	planner.py	Repeats process for drop-off navigation

5. Pathfinding Algorithms (AI Search Module)

The system implements five classical search algorithms, each demonstrating a different search strategy.

5.1 Breadth-First Search (BFS)

- Explores the grid level by level.
- Uses a **Queue (FIFO)** data structure.
- Guarantees the shortest path in an unweighted grid.
- Computationally expensive due to blind exploration.

5.2 Depth-First Search (DFS)

- Explores one path deeply before backtracking.
- Uses a **Stack (LIFO)** data structure.
- Memory efficient in some cases.
- Does not guarantee optimal paths.

5.3 Uniform Cost Search (UCS)

- Expands the node with the lowest accumulated cost.
- Uses a **Priority Queue** ordered by path cost.
- Guarantees optimal paths in weighted graphs.
- Slower due to uniform expansion.

5.4 Greedy Best-First Search

- Expands the node closest to the goal using heuristic estimation.
- Uses a **Priority Queue** ordered by heuristic value.
- Very fast but not optimal.

5.5 A* Search (Recommended)

- Combines path cost and heuristic estimation.
- Evaluation function: **f(n) = g(n) + h(n)**.

- Guarantees optimal paths while remaining efficient.

Algorithm Comparison

Algorithm	Optimal	Speed	Strategy
BFS	Yes	Slow	Blind radial expansion
DFS	No	Fast (sometimes)	Deep exploration
UCS	Yes	Slow	Lowest cost first
Greedy	No	Very Fast	Heuristic driven
A*	Yes	Fast	Cost + heuristic

6. Intelligent Reasoning & Scheduling Modules

6.1 Knowledge Base (Propositional Logic)

- Stores facts such as battery status.
- Applies **Forward Chaining** to infer new conclusions.
- Enables reasoning-based decisions (e.g., need to recharge).

6.2 Constraint Satisfaction Problem (CSP)

The CSP module solves the task scheduling problem.

Components:

- **Variables:** Selected tasks (Clean, Tea, Charge).
- **Domains:** Time slots (1, 2, 3, ...).
- **Constraints:**
    - All-Different constraint.
    - Precedence constraints (Charge before Tea).

Solver:

- Uses **Backtracking Search** to find a valid schedule.

Example Output:

- Charge → 1
- Serve Tea → 2
- Clean House → 3

6.3 Automated Execution

Once a valid schedule is found:

- Tasks are displayed in execution order.
- The robot performs them automatically.

7. Battery Management System

- Each movement consumes **1% battery**.
- Robot stops when battery reaches zero.
- Recharge task restores battery to **100%**.
- Battery level is continuously displayed in the GUI.

8. Graphical User Interface (GUI)

The GUI provides:

- Real-time grid visualization.
- Smooth robot movement animations.
- Path overlays for navigation.
- Interactive controls for task selection and algorithm choice.

9. Output Screenshots

Figure 1: Home Service Robot – Initial Environment Setup

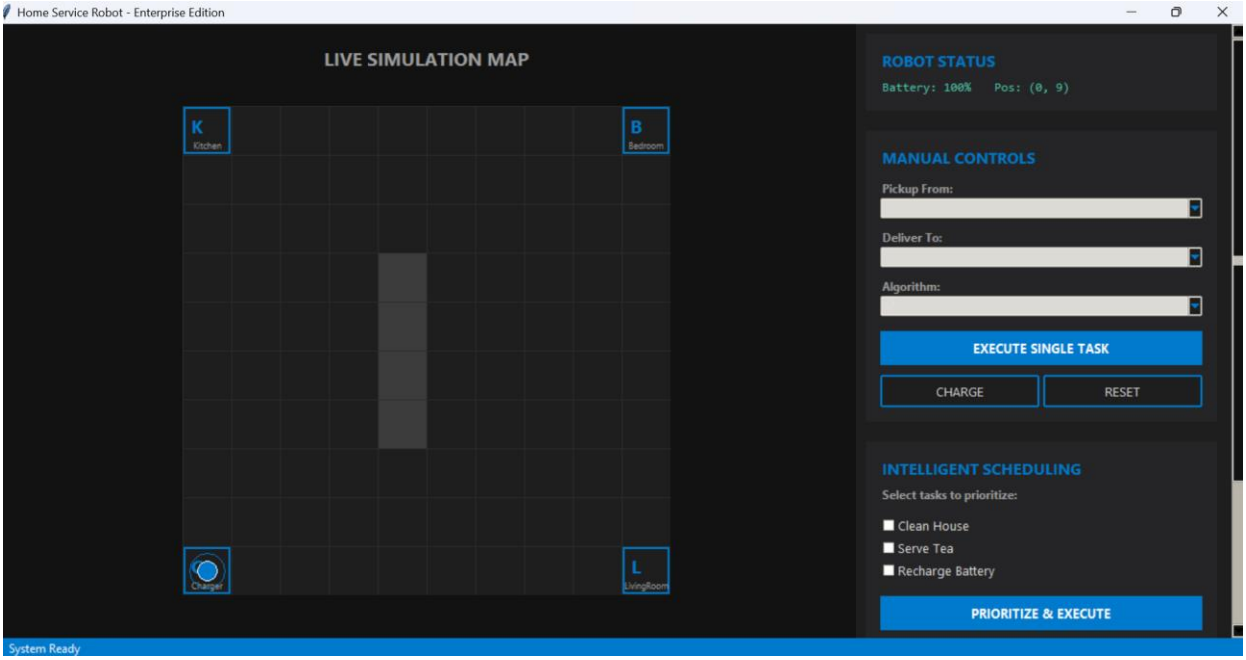


Figure 2: Manual Task Selection and Algorithm Choice

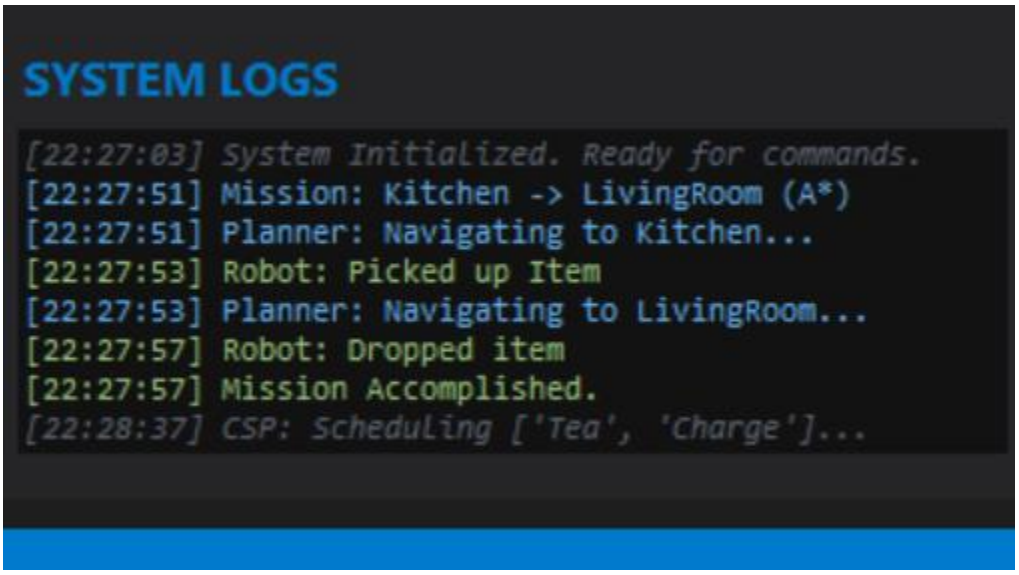


Figure 3: Robot Navigation Using A* Algorithm

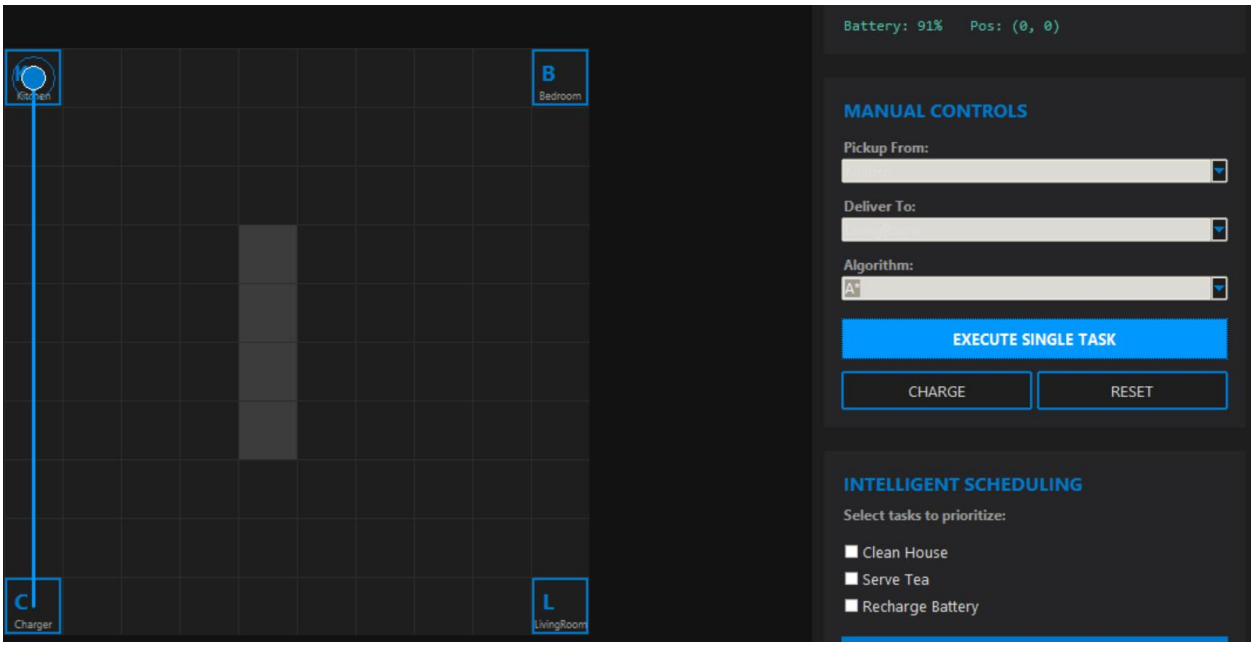


Figure 4: Intelligent Task Scheduling (CSP Output)

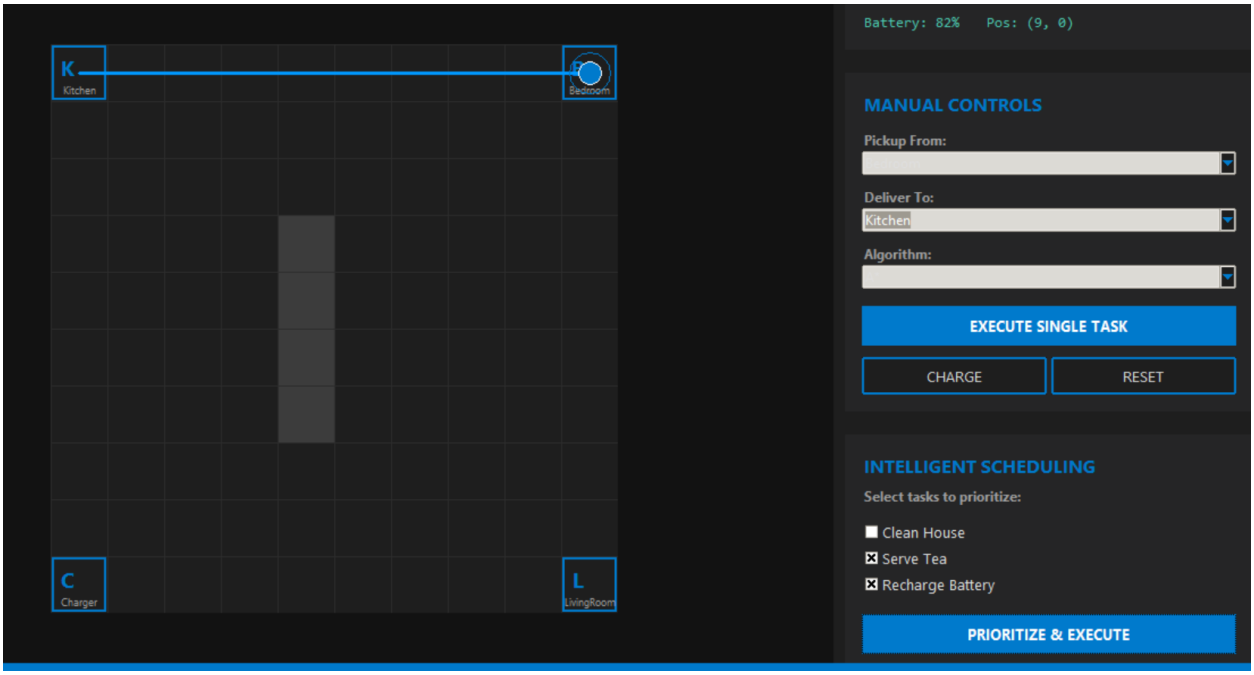
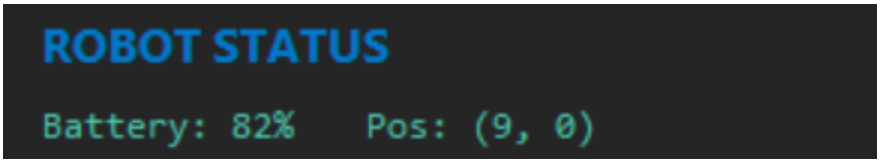


Figure 5: Battery Drain and Recharge Visualization



10. File Structure Summary

- **gui.py** – Handles user interaction, visualization, and animation.
- **planner.py** – Decomposes tasks and manages execution flow.
- **search.py** – Implements AI search algorithms.
- **robot.py** – Models robot state (position, battery, inventory).
- **environment.py** – Represents the grid world, locations, and obstacles.
- **csp.py** – Solves task scheduling using backtracking.
- **knowledge_base.py** – Implements logical reasoning and inference.

11. Conclusion

This project successfully demonstrates how multiple Artificial Intelligence concepts can be integrated into a single, coherent system. By combining search algorithms, logical reasoning, and constraint satisfaction with an interactive GUI, the Home Service Robot simulation provides both educational value and practical insight into autonomous intelligent systems.